

[开始使用](#)

迁移至 GLM-5.1

[Copy page](#)

💡 本文介绍如何将调用从 GLM-4.5 GLM-4.6 GLM-4.7 或其它早期模型迁移到我们迄今为止最强的编码模型 Z.ai GLM-5.1，涵盖采样参数差异、流式工具调用等要点。

GLM-5.1 的特性

- 支持更大上下文与输出：最大上下文 200K，最大输出 128K。
- 新增支持工具调用过程的流式输出（`tool_stream=true`），实时获取工具调用参数。
- 同 GLM-4.7 系列，支持深度思考（`thinking={ type: "enabled" }`），当开启时，为强制深度思考。
- 更卓越的代码性能和先进的推理能力。

迁移清单（Checklist）

- ☐ 更新模型编码为 `glm-5.1`
- ☐ 采样参数：`temperature` 默认值 `1.0`，`top_p` 默认值 `0.95`，建议只选一个进行调参
- ☐ 深度思考：按需关闭或启用 `thinking={ type: "enabled" }`，用于复杂推理/编码
- ☐ 流式响应：启用 `stream=true` 并正确处理 `delta.reasoning_content` 与 `delta.content`
- ☐ 流式工具调用：启用 `stream=true` 和 `tool_stream=true` 并流式拼接 `delta.tool_calls[*].function.arguments`
- ☐ 最大输出与上下文：合理设置 `max_tokens`（GLM-5.1 最大输出 128K，上下文 200K）

>

开始迁移

1. 更新模型编码

- 将 `model` 更新为 `glm-5.1`。

```
resp = client.chat.completions.create(  
    model="glm-5.1",  
    messages=[{"role": "user", "content": "简述 GLM-5 的优势"}]  
)
```

2. 更新采样参数

- `temperature`：控制随机性；数值更高更发散，数值更低更稳定。
- `top_p`：控制核采样；更高值扩大候选集，更低值收敛候选集。
- `temperature` 默认为 `1.0`，`top_p` 默认为 `0.95`，不建议同时调整两者。

```
# Plan A: 使用 temperature (推荐)  
resp = client.chat.completions.create(  
    model="glm-5.1",  
    messages=[{"role": "user", "content": "写一段更具创意的品牌介绍"}],  
    temperature=1.0  
)  
  
# Plan B: 使用 top_p  
resp = client.chat.completions.create(  
    model="glm-5.1",  
    messages=[{"role": "user", "content": "生成更稳定的技术说明"}],  
    top_p=0.8  
)
```



- GLM-5.1 延续支持深度思考能力，默认为开启。
- 在复杂推理、编码任务中，建议开启：

```
resp = client.chat.completions.create(  
    model="glm-5.1",  
    messages=[{"role": "user", "content": "为我设计一个三层微服务架构"}],  
    thinking={"type": "enabled"}  
)
```

4. 流式输出与流式工具调用 (可选)

- GLM-5.1 支持工具调用过程的实时流式构建与输出，默认 `False` 关闭，需同时打开：
 - `stream=True` : 开启响应的流式输出
 - `tool_stream=True` : 开启工具调用参数的流式输出



```
response = client.chat.completions.create(
    model="glm-5.1",
    messages=[{"role": "user", "content": "北京天气怎么样"}],
    tools=[
        {
            "type": "function",
            "function": {
                "name": "get_weather",
                "description": "获取指定地点当前的天气情况",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "location": {"type": "string", "description": "城市，例如：北京"},
                        "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
                    },
                    "required": ["location"]
                }
            }
        }
    ],
    stream=True,
    tool_stream=True,
)

# 初始化流式收集变量
reasoning_content = ""
content = ""
final_tool_calls = {}
reasoning_started = False
content_started = False

# 处理流式响应
for chunk in response:
    if not chunk.choices:
        continue

    delta = chunk.choices[0].delta

    # 流式推理过程输出
    if hasattr(delta, 'reasoning_content') and delta.reasoning_content:
```

```
if not reasoning_started and delta.reasoning_content.strip():
```

```
    print("\n🧠 思考过程: ")
```

```
    reasoning_started = True
```

```
    reasoning_content += delta.reasoning_content
```

```
    print(delta.reasoning_content, end="", flush=True)
```

```
# 流式回答内容输出
```

```
if hasattr(delta, 'content') and delta.content:
```

```
    if not content_started and delta.content.strip():
```

```
        print("\n💬 回答内容: ")
```

```
        content_started = True
```

```
        content += delta.content
```

```
        print(delta.content, end="", flush=True)
```

```
# 流式工具调用信息（参数拼接）
```

```
if delta.tool_calls:
```

```
    for tool_call in delta.tool_calls:
```

```
        idx = tool_call.index
```

```
        if idx not in final_tool_calls:
```

```
            final_tool_calls[idx] = tool_call
```

```
            final_tool_calls[idx].function.arguments = tool_call.function.arguments
```

```
        else:
```

```
            final_tool_calls[idx].function.arguments += tool_call.function.arguments
```

```
# 输出最终的工具调用信息
```

```
if final_tool_calls:
```

```
    print("\n📋 命中 Function Calls :")
```

```
    for idx, tool_call in final_tool_calls.items():
```

```
        print(f"    {idx}: 函数名: {tool_call.function.name}, 参数: {tool_call.function.arguments}")
```

详见: [工具流式输出文档](#)

5. 测试与回归

在开发环境中先行验证迁移后的调用是否稳定，关注：

- 响应是否符合预期、是否出现过度随机或过度保守的输出



>

更多资源



核心参数

模型常见参数概念与采样建议



工具流式输出

查看工具流式输出使用详情



API 参考

查看完整的 API 文档



技术支持

获取技术支持和帮助